

Competitive Programming

Section 31: Sieve, Primes, and Factorization

Competitive Programming Course

2026-07-06

Table of contents

1	Why sieves matter	3
2	Sieve of Eratosthenes	4
3	Prefix prime counts	5
4	Smallest prime factor table	6
5	Divisor count formula	7
6	Worked example	8
7	C++ notes	9
8	Python notes	10
9	Common mistakes	11
10	Submission checklist	12

1 Why sieves matter

Trial division answers one number at a time. A sieve answers many questions up to a fixed limit after preprocessing.

Use a sieve when the statement has:

```
many queries  
all numbers  $\leq N$   
prime counts, primality, divisor counts, or repeated factorization
```

2 Sieve of Eratosthenes

Start by assuming every number at least 2 is prime. Then for each prime p , mark multiples of p as composite.

Why start at $p \cdot p$?

$2p, 3p, \dots, (p-1)p$

already have a smaller prime factor and were marked earlier.

Pseudocode:

```
is_prime[0] = is_prime[1] = false
for p in 2..sqrt(n):
    if is_prime[p]:
        for x in p*p, p*p+p, ... <= n:
            is_prime[x] = false
```

Complexity: $O(n \log \log n)$.

3 Prefix prime counts

After computing `is_prime`, range queries are just prefix sums:

```
pref[i] = number of primes <= i  
answer(l,r) = pref[r] - pref[l-1]
```

4 Smallest prime factor table

For factorization queries, store:

```
spf[x] = smallest prime factor of x
```

Initialize `spf[x] = x`. When a prime `p` is found, fill unassigned multiples:

```
for x in p*p, p*p+p, ...:
    if spf[x] == x:
        spf[x] = p
```

Then factor a number by repeated division:

```
while x > 1:
    p = spf[x]
    exponent = 0
    while x % p == 0:
        exponent += 1
        x /= p
```

This is fast because each division removes a prime factor.

5 Divisor count formula

If:

$$x = p_1^{e_1} * p_2^{e_2} * \dots * p_k^{e_k}$$

then every divisor chooses an exponent from $0 \dots e_i$ for each prime. Therefore:

$$\text{number_of_divisors} = (e_1+1)(e_2+1)\dots(e_k+1)$$

Example:

$$\begin{aligned} 36 &= 2^2 * 3^2 \\ \text{divisors} &= (2+1)(2+1) = 9 \end{aligned}$$

6 Worked example

Factor 360:

```
360 = 2^3 * 3^2 * 5^1  
distinct prime factors = 3  
total prime factors with multiplicity = 3 + 2 + 1 = 6  
largest prime factor = 5  
number of divisors = (3+1)(2+1)(1+1) = 24
```

7 C++ notes

`vector<bool>` is compact and fine for primality flags. For `spf`, use `vector<int>`.

Use `long long` for `p*p` in loops:

```
for (long long p = 2; p * p <= n; p++)
```

This avoids overflow when the limit grows in harder problems.

8 Python notes

Python lists are fine up to around 10^6 or a few million.

For large sieves, slice assignment can be faster, but the explicit loop is clear and matches C++.

9 Common mistakes

```
Treating 1 as prime.  
Starting multiples at 2*p and doing extra work.  
Forgetting prefix[l-1] in range counts.  
Using largest prime factor when SPF is required.  
Not handling x = 1 in factorization.  
Computing divisor count as sum of exponents instead of product of exponent+1.
```

10 Submission checklist

1. What is the maximum number that appears?
2. Do you need primality only or factorization?
3. Are 0 and 1 marked non-prime?
4. Are range queries answered with prefix counts?
5. Does factorization handle $x = 1$?